Guia de Reprodutibilidade dos Experimentos
Detecção de Anomalias em Subespaços

Version Versão 1.0

Wagner Carlos Mariani

September 16, 2026

Contents

1 Descrição geral do experimento **4**
- 1.1 Ambiente sintético controlado . . . 4
- 1.2 Conjuntos de treino, validação e teste . . . 5
- 1.3 Partição das variáveis . . . 6
- 1.4 Comparação entre avaliação global e avaliação local . . . 7
- 1.5 Representação residual comum às estratégias global e local . 8
- 1.6 Operadores de score . . . 9
- 1.7 Diagnóstico baseado exclusivamente em dados normais . . . 10
- 1.8 Autoencoder recorrente . . . 13
- 1.9 Validação, limiares e ataques . . . 14
- 1.10 Experimento prospectivo da fronteira estrutural com RNN . . 15

2 Organização dos diretórios **16**
- 2.1 Diretório `dados` . . . 16
  - 2.1.1 Fronteira estrutural . . . 17
  - 2.1.2 Experimento RNN A06 . . . 18
  - 2.1.3 Fronteira prospectiva com RNN . . . 18
- 2.2 Diretório `resultado` . . . 19
  - 2.2.1 Etapas exploratórias iniciais . . . 20
  - 2.2.2 Experimento confirmatório . . . 21
  - 2.2.3 Fronteira estrutural . . . 21
  - 2.2.4 Experimento RNN A06 . . . 22
  - 2.2.5 Fronteira prospectiva com RNN . . . 23
- 2.3 Diretório `codigo` . . . 24

3 Roadmap para reprodução dos experimentos **25**
- 3.1 Preparação do ambiente Python . . . 26
- 3.2 Reprodução da etapa exploratória . . . 27
- 3.3 Reprodução da etapa confirmatória . . . 27
- 3.4 Preparação da fronteira estrutural . . . 28
- 3.5 Descoberta dos grupos . . . 28
- 3.6 Diagnóstico estrutural com dados normais . . . 29
- 3.7 Análises de referência . . . 30
- 3.8 Reprodução do experimento inicial com RNN no cenário A06 30
- 3.9 Registro do protocolo da fronteira com RNN . . . 31
- 3.10 Treinamento das RNNs . . . 31

3.11 Diagnóstico dos resíduos da RNN 32
3.12 Registro de integridade antes dos testes 32
3.13 Teste global versus local 33

4 Programas e scripts **34**
4.1 gerar_normal_sintetico.py 34
4.2 gerar_teste_ataque_piloto.py 35
4.3 auditar_ataques_piloto.py 35
4.4 auditar_correlacoes_normal.py 36
4.5 auditar_ranges_fisicos.py 36
4.6 descobrir_grupos_normal.py 36
4.7 baseline_mahalanobis_global_local.py 37
4.8 baselines_estaticos_global_local.py 37
4.9 diagnostico_mahalanobis_fpr_casado.py 38
4.10 congelar_grade_exploratoria.py 38
4.11 executar_grade_exploratoria.py 39
4.12 congelar_eventos_confirmatorios.py 39
4.13 congelar_confirmacao.py 40
4.14 gerar_realizacoes_confirmatorias.py 40
4.15 descobrir_grupos_confirmatorios.py 40
4.16 auditar_realizacoes_confirmatorias.py 41
4.17 analisar_estabilidade_confirmacao.py 41
4.18 analisar_estabilidade_primaria.py 41
4.19 diagnosticar_vantagem_normal_only.py 42
4.20 executar_confirmacao_limpo.py 42
4.21 executar_confirmacao.py 43
4.22 registrar_grade_estrutural.py 44
4.23 gerar_cenarios_estruturais.py 44
4.24 auditar_cenarios_estruturais.py 44
4.25 analisar_descoberta_grupos_fronteira.py 45
4.26 diagnosticar_fronteira_normal_only.py 45
4.27 diagnosticar_fronteira_oraculo.py 45
4.28 executar_probes_fronteira_estrutural.py 46
4.29 executar_probes_fronteira_oraculo.py 47
4.30 analisar_direcoes_b05_b06.py 47
4.31 registrar_protocolo_rnn_fronteira.py 48
4.32 codigo/codigo/registrar_protocolo_rnn_a06.py . . . 48
4.33 treinar_rnn_a06_residuos_normais.py 49

4.34 diagnosticar_rnn_a06_residuos_normal_only.py . . . 49
4.35 testar_rnn_a06_global_local.py 50
4.36 treinar_rnn_fronteira_residuos_normais.py 50
4.37 diagnosticar_rnn_fronteira_residuos_normal_only.py 50
4.38 testar_rnn_fronteira_global_local.py 51
4.39 analisar_fronteira_grade.py 52
4.40 auditar_plausibilidade_univariada.py 52
4.41 gerar_normal_estrutural.py 52
4.42 testar_regressao_gerador_estrutural.py 53

5 Integridade e reprodutibilidade **54**
5.1 Sementes e geração determinística 54
5.2 Congelamento dos protocolos 55
5.3 Separação entre dados normais e ataques 55
5.4 Integridade da etapa confirmatória 56
5.5 Integridade da fronteira estrutural 56
5.6 Congelamento da bateria prospectiva com RNN 57
5.7 Status do cenário A06 58
5.8 Sequência de integridade 58

Lista de Códigos

1 Descrição geral do experimento

Este conjunto de experimentos foi desenvolvido para investigar em quais condições a avaliação de anomalias em subespaços de variáveis pode apresentar vantagem em relação à avaliação realizada sobre o espaço completo de atributos.

O objetivo dos experimentos não é demonstrar que a decomposição de um sistema em grupos seja sempre vantajosa. Ao contrário, o ambiente experimental foi construído para representar diferentes condições estruturais, incluindo situações nas quais a decomposição é favorável, situações neutras e situações nas quais sua utilidade depende da geometria da anomalia observada.

A motivação central é a possibilidade de que uma anomalia localizada em um subconjunto de variáveis apresente menor contraste quando sua evidência é avaliada conjuntamente com um grande número de dimensões. Entretanto, a existência dessa vantagem não depende apenas da redução dimensional. Ela depende da interação entre a estrutura estatística dos dados, a partição utilizada, a direção da perturbação e o operador empregado para transformar desvios ou resíduos em um score de anomalia.

1.1 Ambiente sintético controlado

Foi construído um ambiente sintético contendo 400 variáveis de processo. No gerador, essas variáveis são organizadas em 20 grupos de 20 variáveis, sendo cada grupo composto por 14 variáveis contínuas e 6 variáveis binárias.

A estrutura estatística dos dados pode ser modificada por meio de parâmetros que controlam, entre outros aspectos, a intensidade das dependências entre variáveis pertencentes ao mesmo grupo e das dependências existentes entre grupos distintos.

Essa característica permite produzir diferentes cenários mantendo constante o número total de variáveis e modificando principalmente sua organização estatística.

Os cenários utilizados na investigação foram organizados em duas famílias, denominadas A e B.

Na família A, a estrutura intragrupo é progressivamente fortalecida enquanto

a dependência entre grupos permanece baixa. Na família B, a relação entre dependências intragrupo e intergrupo é modificada progressivamente, tornando a separação dos grupos menos evidente a partir dos dados normais.

Essa construção produz uma fronteira estrutural controlada e permite observar o comportamento dos métodos em condições distintas, em vez de selecionar apenas um exemplo favorável à decomposição.

1.2 Conjuntos de treino, validação e teste

Para cada cenário estrutural são utilizados três conjuntos de dados distintos: treinamento, validação e teste. A frequência de amostragem adotada no ambiente sintético é de uma amostra por segundo (1 Hz).

O conjunto de treinamento contém 20.000 amostras normais, correspondentes a aproximadamente 5 horas, 33 minutos e 20 segundos de observação. Esse conjunto é utilizado para o treinamento da RNN e para o ajuste das distribuições e matrizes de covariância empregadas pelos detectores.

O conjunto de validação contém 10.000 amostras normais, correspondentes a aproximadamente 2 horas, 46 minutos e 40 segundos. Ele permanece separado do treinamento e é utilizado para acompanhamento da perda de validação, seleção do melhor checkpoint e calibração dos limiares de detecção.

O conjunto de teste contém 20.000 amostras, correspondentes a aproximadamente 5 horas, 33 minutos e 20 segundos. Inicialmente, esse conjunto representa somente o comportamento normal do sistema. Os ataques sintéticos são introduzidos posteriormente e exclusivamente nesse conjunto, depois que o treinamento, a validação, os limiares e o diagnóstico estrutural já foram definidos.

A Tabela 1 resume essa organização.

Table 1: Organização dos conjuntos utilizados em cada cenário sintético.

Conjunto	**Amostras**	**Amostras após janela**	**Finalidade principal**
Treinamento	20.000	19.941	Treinamento e ajuste dos detectores
Validação	10.000	9.941	Checkpoint e limiares
Teste	20.000	19.941	Avaliação final com ataques

A diferença entre o número original de amostras e o número efetivamente avaliável pela RNN decorre do uso de janelas temporais de 60 amostras. Para uma sequência contendo N observações, são produzidas

$$N - 60 + 1 = N - 59 \tag{1}$$

janelas válidas. Como o score é associado ao último instante de cada janela, as primeiras 59 observações de cada conjunto não possuem um score correspondente.

Consequentemente, o treinamento com 20.000 observações produz 19.941 vetores de resíduos, enquanto as 10.000 observações de validação produzem 9.941 vetores de resíduos. A implementação verifica explicitamente essas dimensões antes de prosseguir com o diagnóstico.

No conjunto de teste são inseridos 20 eventos de ataque, cada um com duração de 100 segundos. Dessa forma, cada probe contém 2.000 amostras atacadas. O protocolo experimental fixa a duração dos eventos e determina que os ataques sejam utilizados somente no teste, após o treinamento, o ajuste das covariâncias e a calibração dos limiares.

Após o alinhamento temporal da RNN, o conjunto de teste utilizado para o cálculo das métricas possui 19.941 instantes avaliáveis, dos quais 2.000 correspondem aos eventos de ataque e 17.941 ao comportamento normal.

É importante observar que treinamento e validação permanecem integralmente normais. Os rótulos e perturbações de ataque são empregados somente na etapa final de teste.

1.3 Partição das variáveis

Embora o gerador sintético possua uma divisão conhecida das variáveis, essa informação não é utilizada diretamente no procedimento principal de descoberta dos grupos.

A partição candidata é obtida exclusivamente a partir dos dados normais de treinamento. Inicialmente, são calculadas as correlações absolutas de Pearson entre as variáveis. Os valores das correlações entre pares são então separados automaticamente em dois conjuntos por meio de KMeans, correspondentes às relações de menor e maior intensidade. A partir dessa

separação é definido o limiar de conectividade utilizado pelo procedimento de agrupamento.

Com base nesse limiar é construído um grafo no qual as variáveis representam os vértices e as relações consideradas suficientemente fortes representam arestas. As componentes conexas resultantes são utilizadas como grupos candidatos.

Os nomes das variáveis e os identificadores dos grupos utilizados internamente pelo gerador não participam desse procedimento.

A estrutura conhecida do gerador pode ser utilizada posteriormente como referência para avaliar a qualidade da partição descoberta. Essa separação permite distinguir dois problemas diferentes: descobrir uma estrutura candidata a partir de dados normais e avaliar se determinada estrutura é efetivamente útil para a detecção de anomalias.

A expressão *partição de referência* deve ser entendida como a organização utilizada pelo gerador para construir e controlar os experimentos, e não necessariamente como uma estrutura que possa ser recuperada a partir dos dados normais em qualquer cenário. Em configurações nas quais o mecanismo gerador reduz ou elimina a informação estatística que distingue os grupos, essa partição pode deixar de ser identificável a partir das observações normais. Nesses casos, sua utilização permanece válida como referência experimental para controlar onde as perturbações são aplicadas e para realizar análises de comparação, mas não como informação disponível ao procedimento de descoberta.

1.4 Comparação entre avaliação global e avaliação local

A comparação principal considera duas maneiras de transformar a mesma informação em um score de anomalia.

Na abordagem global, todas as variáveis são avaliadas conjuntamente.

Na abordagem local, as mesmas variáveis são separadas de acordo com uma partição candidata. Nas comparações baseadas em Mahalanobis, um score é calculado separadamente para cada grupo a partir do respectivo subconjunto do vetor avaliado. Esses scores são então combinados por meio do máximo entre os grupos, produzindo um único score local por instante. O limiar local é calculado posteriormente sobre essa distribuição já fusionada de scores,

utilizando o percentil 99 da validação normal. Portanto, a estratégia local emprega um único limiar sobre o máximo dos scores dos grupos, e não um limiar independente para cada grupo.

Nos experimentos realizados com autoencoder recorrente, essa comparação possui uma característica importante: não são treinados autoencoders independentes para os diferentes grupos.

Seja $S_{k,t}$ o score quadrático de Mahalanobis associado ao grupo G_k no instante t. O score local utilizado para a decisão é definido por

$$S_{L,t} = \max_k S_{k,t}, \tag{2}$$

e seu limiar é obtido exclusivamente sobre a validação normal por

$$\tau_L = P_{99}\left(S_{L,\text{val}}\right). \tag{3}$$

De forma análoga, o score global $S_{G,t}$ é calculado utilizando simultaneamente todas as dimensões do vetor de resíduos, e seu limiar é dado por

$$\tau_G = P_{99}\left(S_{G,\text{val}}\right). \tag{4}$$

Um caso limite importante ocorre quando a partição candidata contém apenas um grupo com todas as variáveis, isto é, $K = 1$. Nessa situação, as estratégias local e global utilizam exatamente o mesmo conjunto de dimensões para ajustar o modelo de Mahalanobis, de modo que seus scores e limiares coincidem, salvo efeitos puramente numéricos de implementação. Esse caso funciona como controle interno do experimento: sem uma decomposição efetiva do espaço, não deve existir diferença atribuível à avaliação local.

1.5 Representação residual comum às estratégias global e local

Uma única RNN é treinada utilizando simultaneamente todas as 400 variáveis. O modelo gera um vetor de resíduos que é posteriormente utilizado tanto pela estratégia global quanto pela estratégia local.

Seja

$$\mathbf{e}_t = \mathbf{x}_t - \hat{\mathbf{x}}_t \tag{5}$$

o vetor de resíduos produzido no instante t, onde $\mathbf{x}_t$ representa a observação original e $\hat{\mathbf{x}}_t$ sua reconstrução.

Na estratégia global, o score é calculado utilizando todas as dimensões de $\mathbf{e}_t$.

Na estratégia local, o mesmo vetor é particionado segundo os grupos

$$G_1, G_2, \ldots, G_K, \tag{6}$$

e cada subconjunto $\mathbf{e}_{G_k,t}$ é avaliado separadamente.

Consequentemente, as estratégias global e local utilizam exatamente a mesma RNN e os mesmos resíduos. A diferença experimental encontra-se na forma como esses resíduos são agregados para produzir o score de anomalia.

Essa configuração permite separar os efeitos decorrentes da avaliação local daqueles que poderiam ser causados pelo treinamento de modelos com capacidades diferentes.

1.6 Operadores de score

Durante a investigação foram avaliadas diferentes formas de transformar desvios ou resíduos em scores de anomalia.

Entre os operadores analisados estão a Energia, definida nos experimentos estáticos como a média dos desvios padronizados elevados ao quadrado, o maior desvio individual (MAX), agregações baseadas nos maiores desvios por meio de TOP-k e o score quadrático de Mahalanobis. Nos experimentos com RNN, o erro quadrático médio (MSE) é utilizado como função de perda de reconstrução durante o treinamento e não deve ser confundido com o operador estático Energia.

Os operadores foram utilizados para verificar se a eventual vantagem da decomposição poderia ser explicada simplesmente pela diminuição do número de dimensões avaliadas.

Os resultados mostraram que o efeito da divisão depende do operador empregado. Em particular, um operador como MAX pode apresentar comportamento global e local equivalente em determinadas condições, enquanto outros operadores podem ser fortemente afetados pela estrutura dos subespaços.

O score quadrático de Mahalanobis recebeu atenção especial nas etapas posteriores, pois considera a estrutura de covariância dos vetores avaliados e permite estabelecer uma relação matemática entre a avaliação global e a avaliação realizada nos subespaços.

Nos experimentos correspondentes, as matrizes de covariância são estimadas por meio do estimador Ledoit–Wolf. Os modelos não assumem que os vetores avaliados estejam centrados em zero: o centro da distribuição é estimado a partir dos dados normais de treinamento e utilizado no cálculo dos scores quadráticos de Mahalanobis, tanto no caso global quanto nos modelos locais. Nos experimentos com RNN, esses vetores correspondem aos resíduos de reconstrução; nos experimentos estáticos, correspondem à representação utilizada pelo respectivo detector.

1.7 Diagnóstico baseado exclusivamente em dados normais

Uma das etapas centrais da investigação consiste em avaliar uma partição candidata antes da observação de qualquer ataque.

Para isso, são utilizados exclusivamente dados normais de treinamento e validação.

A partir dos scores obtidos na validação são estimados os limiares utilizados pelas estratégias global e local. Também são calculadas medidas relacionadas à distância existente entre o comportamento normal típico e esses limiares.

Para o caso global, pode-se representar essa margem por

$$\text{gap}_G = \tau_G - \mu_G, \tag{7}$$

onde τ_G representa o limiar global e μ_G representa a média dos scores globais obtidos na validação normal. Essa quantidade não deve ser confundida

com o vetor médio dos resíduos estimado internamente pelo modelo de Mahalanobis durante o ajuste sobre os dados normais de treinamento.

Para cada grupo G_k, a margem local é calculada em relação ao limiar da estratégia local já fusionada por máximo,

$$\mathrm{gap}_{L,k} = \tau_L - \mu_{L,k}, \tag{8}$$

onde $\mu_{L,k}$ representa a média dos scores quadráticos de Mahalanobis do grupo G_k na validação normal e τ_L é o percentil 99 da distribuição do máximo entre os scores dos grupos.

$$B_k = \frac{\mathrm{gap}_G}{\mathrm{gap}_{L,k}}. \tag{9}$$

A componente geométrica do diagnóstico compara a matriz de precisão estimada globalmente, restrita às dimensões do grupo G_k, com a matriz de precisão estimada diretamente nesse grupo. Denotando essas matrizes por $\mathbf{P}_G^{(k)}$ e $\mathbf{P}_{L,k}$, respectivamente, são calculados os autovalores generalizados do problema

$$\mathbf{P}_G^{(k)}\mathbf{v} = r\,\mathbf{P}_{L,k}\mathbf{v}. \tag{10}$$

Para uma direção $\mathbf{v}$ pertencente ao subespaço do grupo, a razão correspondente pode ser interpretada como

$$r(\mathbf{v}) = \frac{\mathbf{v}^\top \mathbf{P}_G^{(k)} \mathbf{v}}{\mathbf{v}^\top \mathbf{P}_{L,k} \mathbf{v}}. \tag{11}$$

Sejam $r_{\min,k}$, $r_{\mathrm{med},k}$ e $r_{\max,k}$, respectivamente, o menor, o mediano e o maior autovalor generalizado obtidos para o grupo. Os scripts calculam então

$$F_{\mathrm{pess},k} = \frac{B_k}{r_{\max,k}}, \tag{12}$$

$$F_{\mathrm{med},k} = \frac{B_k}{r_{\mathrm{med},k}}, \tag{13}$$

e

$$F_{\text{opt},k} = \frac{B_k}{r_{\min,k}}. \tag{14}$$

O fator pessimista utiliza a direção geometricamente mais desfavorável à avaliação local. Assim, valores $F_{\text{pess},k} > 1$ caracterizam, dentro da aproximação quadrática utilizada pelo diagnóstico, uma condição conservadora favorável à avaliação local para todas as direções consideradas no subespaço. Essa condição se refere à relação entre a geometria dos scores e suas margens de decisão e não deve ser interpretada, isoladamente, como previsão da magnitude de ganho em métricas empíricas como recall ou F1.

Valores $F_{\text{pess},k} \leq 1$ não significam que a estratégia local necessariamente apresentará desempenho inferior. Nesses casos, o critério apenas deixa de fornecer uma garantia conservadora válida para todas as direções, e o resultado pode depender da direção concreta na qual a perturbação se manifesta.

A combinação dos fatores pessimista e otimista permite organizar o diagnóstico em três regiões teóricas. Quando $F_{\text{pess},k} > 1$, a condição favorável à avaliação local é satisfeita para todas as direções consideradas no subespaço, dentro da aproximação utilizada. Quando $F_{\text{pess},k} \leq 1 \leq F_{\text{opt},k}$, existem direções potencialmente favoráveis e outras para as quais a condição não é garantida, caracterizando uma região dependente da direção da perturbação. Finalmente, quando $F_{\text{opt},k} < 1$, nem mesmo a direção mais favorável satisfaz a condição dentro dessa aproximação. Os casos de igualdade correspondem aos limites entre essas regiões e devem ser interpretados como condições de transição, e não como evidência de vantagem ou desvantagem empírica.

O fator mediano $F_{\text{med},k}$ é mantido como medida descritiva da geometria intermediária do subespaço, mas não é utilizado para estabelecer as garantias conservadoras definidas pelos fatores pessimista e otimista.

Observação terminológica: neste documento, quando uma condição estrutural é denominada *favorável*, o termo se refere ao resultado do diagnóstico matemático dentro da aproximação utilizada e não, por si só, à observação de maior recall, F1 ou outra métrica no conjunto de teste. O desempenho empírico é avaliado posteriormente com as perturbações sintéticas e constitui uma etapa distinta da classificação estrutural obtida apenas com dados normais.

Todo esse diagnóstico é realizado sem utilizar amostras de ataque.

1.8 Autoencoder recorrente

Para verificar se o fenômeno permanece presente após uma transformação temporal e não linear dos dados, foi utilizado um autoencoder baseado em rede neural recorrente.

O modelo recebe simultaneamente as 400 variáveis e opera sobre janelas temporais de 60 amostras.

O encoder utiliza uma camada `RNN` com 64 unidades ocultas e ativação *tanh*. O estado oculto final produzido pelo encoder é utilizado para inicializar uma segunda camada `RNN`, empregada como decoder. Durante a reconstrução, o decoder recebe uma sequência de vetores nulos no espaço oculto e produz uma sequência de representações com 64 dimensões, posteriormente projetadas por uma camada linear para as 400 variáveis originais.

O treinamento é realizado exclusivamente com dados normais, utilizando o erro quadrático médio como função de perda. Durante o treinamento e a validação, essa perda é calculada entre a sequência reconstruída e a sequência original considerando todos os instantes das janelas temporais de 60 amostras.

Após o treinamento, a construção dos scores de detecção utiliza uma representação mais específica: para cada janela é extraído apenas o vetor de resíduos correspondente ao seu último instante, calculado pela diferença entre as 400 variáveis observadas e reconstruídas nesse instante. Esses vetores de resíduos são armazenados para os conjuntos normais de treinamento e validação e posteriormente utilizados tanto pela estratégia global quanto pela estratégia local.

Os resíduos são mantidos com sinal, isto é, não são convertidos em valores absolutos nem elevados ao quadrado antes do ajuste dos modelos de Mahalanobis. O estimador Ledoit–Wolf é ajustado diretamente sobre esses vetores no caso global e sobre os respectivos subconjuntos de dimensões no caso local.

Assim, a RNN atua como uma representação comum aos dois métodos. A investigação local ocorre posteriormente, no espaço dos resíduos produzidos por essa mesma rede.

1.9 Validação, limiares e ataques

Os limiares de detecção são determinados utilizando exclusivamente dados normais de validação.

Nos experimentos principais com RNN foi utilizado o percentil 99 da distribuição normal dos respectivos scores. Para um score S, o procedimento pode ser representado por

$$\tau = P_{99}\left(S_{\text{val}}\right). \tag{15}$$

A expressão *FPR nominal de 1%* utilizada ao longo deste documento refere-se a essa regra de calibração sobre a validação normal: aproximadamente 1% dos scores de validação encontram-se acima do limiar definido pelo percentil 99. Isso não implica que a taxa de falsos positivos observada posteriormente no conjunto de teste seja necessariamente igual a 1%, pois a distribuição do teste pode diferir da distribuição de validação.

Nenhuma amostra de ataque participa do treinamento da RNN, da descoberta dos grupos, da estimação das matrizes de covariância, da definição dos limiares ou do diagnóstico estrutural realizado sobre os dados normais.

Os ataques sintéticos são introduzidos posteriormente como mecanismo de avaliação empírica das condições estruturais identificadas anteriormente pelo diagnóstico normal-only.

A utilização da estrutura de referência nessa etapa possui finalidade exclusivamente experimental: ela permite construir perturbações localizadas de forma controlada e conhecida. Essa informação não é fornecida ao procedimento principal de descoberta dos grupos nem ao diagnóstico baseado nas partições descobertas. Dessa forma, a estrutura de referência controla onde a intervenção sintética é aplicada, enquanto a estratégia de detecção continua operando com a estrutura obtida a partir dos dados normais.

- P01: m = 8 variáveis e intensidade δ = 1.25;
- P02: m = 8 variáveis e intensidade δ = 1.50.

A posição dos eventos, sua duração e a ordem das perturbações são mantidas constantes entre os diferentes cenários, permitindo comparar o efeito da mudança estrutural dos dados.

1.10 Experimento prospectivo da fronteira estrutural com RNN

A etapa final foi organizada de forma prospectiva em relação à avaliação com ataques: para os onze cenários dessa bateria, o treinamento, os resíduos normais, os diagnósticos normal-only e os respectivos registros de integridade foram concluídos antes da execução dos testes contendo perturbações.

Foram utilizados onze cenários:

$$\{A01, A02, A03, A04, A05, B01, B02, B03, B04, B05, B06\}. \tag{16}$$

Para cada cenário foi treinada uma RNN monolítica e foram produzidos os resíduos normais de treinamento e validação.

Antes da execução dos ataques foram registrados e congelados o protocolo experimental, os checkpoints das RNNs, a ordem das variáveis, os resíduos normais e os diagnósticos produzidos exclusivamente com dados normais.

Para cada conjunto de artefatos relevante foram calculados hashes SHA-256, armazenados em arquivos de manifesto. Esses hashes permitem verificar posteriormente que os arquivos utilizados na etapa de teste correspondem exatamente aos mesmos artefatos produzidos e registrados antes da introdução dos ataques.

Foram gerados manifestos específicos para o protocolo experimental, os artefatos de treinamento, os diagnósticos normal-only e, posteriormente, os resultados finais dos testes.

A localização desses arquivos e a função de cada manifesto são apresentadas na Seção 2, dedicada à organização das pastas e dos artefatos do repositório.

Somente após a criação e verificação desses registros de integridade foram executados os scripts responsáveis pela inserção dos ataques e pela avaliação final.

O cenário A06 não foi incluído entre os onze casos prospectivos porque seu comportamento com RNN já havia sido observado anteriormente durante o desenvolvimento do experimento. Esse cenário foi mantido como referência, evitando apresentar uma observação anterior como se constituísse evidência obtida sob o protocolo prospectivo posteriormente definido.

Esse procedimento permite distinguir claramente a etapa de formulação da hipótese e definição do protocolo experimental da etapa utilizada posteriormente para sua avaliação empírica.

2 Organização dos diretórios

O repositório experimental está organizado em três diretórios principais: `codigo`, `dados` e `resultado`.

De forma resumida, a estrutura principal é:

```
sintetico_subespacos/
|-- codigo/
|-- dados/
`-- resultado/
```

O diretório `codigo` contém os programas responsáveis pela geração dos dados, descoberta dos grupos, execução dos experimentos, treinamento das RNNs, diagnósticos e testes.

O diretório `dados` contém os conjuntos sintéticos utilizados nos experimentos, arquivos de configuração, protocolos previamente registrados, manifestos dos cenários e arquivos utilizados para garantir a reprodutibilidade.

O diretório `resultado` contém as saídas geradas pelos diferentes experimentos, incluindo diagnósticos, partições descobertas, scores, métricas, modelos treinados e resultados finais.

2.1 Diretório dados

O diretório `dados` concentra tanto os conjuntos de entrada quanto os artefatos utilizados para registrar previamente as condições experimentais.

Sua organização principal é:

```
dados/
|-- confirmacao/
|-- fronteira_estrutural/
|-- rnn_a06/
|-- rnn_fronteira/
|-- condicoes_confirmatorias.csv
```

```
|-- eventos_confirmatorios_base.csv
|-- grade_exploratoria.csv
|-- mapa_grupos_verdadeiros.csv
|-- ordem_features_confirmatorias.csv
|-- parametros_fisicos_sensores.csv
|-- protocolo_confirmatorio.json
`-- ...
```

A pasta `confirmacao` contém cinco realizações independentes, identificadas como `R01` até `R05`. Cada realização possui três arquivos:

- `treino_normal.csv`;
- `validacao_normal.csv`;
- `teste_normal_base.csv`.

Esses arquivos foram utilizados na etapa confirmatória anterior à construção da fronteira estrutural.

O arquivo `condicoes_confirmatorias.csv` registra as condições de ataque utilizadas nessa etapa, enquanto `eventos_confirmatorios_base.csv` registra a posição, duração e organização dos eventos utilizados nos testes confirmatórios.

O arquivo `ordem_features_confirmatorias.csv` preserva a ordem das variáveis utilizada durante a geração e aplicação das perturbações.

O arquivo `mapa_grupos_verdadeiros.csv` registra a organização de referência utilizada pelo gerador sintético. Essa informação não é fornecida ao algoritmo principal de descoberta dos grupos, sendo utilizada apenas em análises de referência ou na avaliação posterior da qualidade das partições.

O arquivo `parametros_fisicos_sensores.csv` contém os parâmetros utilizados na transformação das variáveis sintéticas para seus respectivos intervalos de operação.

2.1.1 Fronteira estrutural

A pasta

`dados/fronteira_estrutural/`

contém os arquivos que definem a grade de cenários utilizada para estudar a transição entre diferentes estruturas de dependência.

Os principais arquivos são:

- `grade_estrutural.csv`: definição dos cenários estruturais;
- `manifesto_cenarios_estruturais.csv`: relação entre cada cenário e os respectivos conjuntos de dados;
- `protocolo_estrutural.json`: registro do protocolo utilizado nessa etapa;
- `sha256_protocolo_estrutural.txt`: hash SHA-256 do protocolo registrado.

Os conjuntos correspondentes aos diferentes cenários são armazenados no subdiretório:

`dados/fronteira_estrutural/cenarios/`

2.1.2 Experimento RNN A06

A pasta

`dados/rnn_a06/`

contém o protocolo utilizado no primeiro experimento com RNN aplicado ao cenário A06.

Ela contém:

- `protocolo_rnn_a06.json`;
- `sha256_protocolo_rnn_a06.txt`.

O cenário A06 foi utilizado como experimento anterior à bateria prospectiva e é mantido separadamente para preservar essa distinção metodológica.

2.1.3 Fronteira prospectiva com RNN

A pasta

`dados/rnn_fronteira/`

contém os registros referentes ao experimento prospectivo realizado nos onze cenários da fronteira estrutural.

Os principais arquivos são:

- `protocolo_rnn_fronteira.json`;
- `sha256_protocolo_rnn_fronteira.txt`;
- `sha256_artefatos_treino.txt`;
- `sha256_diagnosticos_normal_only.txt`;
- `sha256_resultados_testes.txt`.

O arquivo `sha256_protocolo_rnn_fronteira.txt` registra o hash do protocolo experimental.

O arquivo `sha256_artefatos_treino.txt` registra os hashes dos checkpoints das RNNs, resíduos normais e arquivos de ordem das variáveis produzidos antes da execução dos ataques.

O arquivo `sha256_diagnosticos_normal_only.txt` registra os hashes dos diagnósticos obtidos exclusivamente com dados normais.

Por fim, `sha256_resultados_testes.txt` registra os hashes das saídas obtidas após a execução dos testes contendo ataques.

Essa sequência permite verificar a ordem metodológica adotada:

$$\text{protocolo} \rightarrow \text{treinamento} \rightarrow \text{diagnóstico normal-only} \rightarrow \text{teste com ataques}. \tag{17}$$

Os arquivos de hash permitem verificar que os artefatos produzidos antes da introdução dos ataques não foram alterados posteriormente.

2.2 Diretório `resultado`

O diretório `resultado` concentra as saídas produzidas ao longo das diferentes etapas da investigação. Sua organização reflete a evolução do experimento, desde as análises exploratórias iniciais até a fronteira prospectiva com RNN.

De forma simplificada, os principais blocos são:

```
resultado/
|-- agrupamento_normal/
|-- grade_exploratoria/
|-- confirmacao/
|-- grupos_confirmatorios/
|-- diagnostico_normal_only/
|-- fronteira_estrutural/
|-- rnn_a06/
`-- rnn_fronteira/
```

Além desses diretórios principais, existem pastas de auditoria e análises auxiliares utilizadas durante o desenvolvimento experimental. Elas são mantidas no repositório para preservar a rastreabilidade do processo, mas não constituem necessariamente etapas que precisem ser executadas para reproduzir o experimento final.

2.2.1 Etapas exploratórias iniciais

As pastas `agrupamento_normal`, `baseline_mahalanobis_global_local`, `baselines_estaticos_global_local` e `grade_exploratoria` correspondem às primeiras etapas utilizadas para verificar a existência do fenômeno investigado.

Nelas estão armazenados, entre outros elementos, os grupos descobertos a partir dos dados normais, scores de teste, comparações entre operadores globais e locais e resultados da grade exploratória.

Também pertencem a essa fase algumas análises auxiliares, como:

- `auditoria_correlacoes`;
- `auditoria_ranges`;
- `auditoria_ataques_piloto`;
- `plausibilidade_univariada`;
- `analise_fronteira_grade`.

Esses diretórios documentam verificações realizadas durante a construção do experimento e ajudam a reconstruir a sequência de decisões adotadas.

2.2.2 Experimento confirmatório

A pasta

`resultado/confirmacao/`

contém os resultados da etapa confirmatória realizada com cinco realizações independentes dos dados normais.

Entre seus arquivos estão os resultados completos das condições confirmatórias, comparações primárias entre métodos e medidas de estabilidade dos resultados.

As partições descobertas separadamente para as realizações R01 até R05 são armazenadas em:

`resultado/grupos_confirmatorios/`

O diretório

`resultado/diagnostico_normal_only/`

contém os diagnósticos estruturais calculados exclusivamente a partir dos dados normais dessa etapa.

Arquivos cujo nome contém o termo `parcial` correspondem a saídas intermediárias produzidas durante a execução e preservadas como mecanismo de checkpoint e rastreabilidade. Os arquivos sem esse sufixo correspondem às saídas finais da respectiva etapa.

2.2.3 Fronteira estrutural

A pasta

`resultado/fronteira_estrutural/`

reúne os resultados produzidos para os cenários A01–A06 e B01–B06.

Sua organização principal é:

```
fronteira_estrutural/
|-- auditoria_normal/
|-- descoberta_grupos/
|-- diagnostico_normal_only/
|-- diagnostico_oraculo/
```

```
|-- probes/
|-- probes_oraculo/
`-- analise_direcoes/
```

A pasta `descoberta_grupos` contém uma subpasta para cada cenário e o respectivo arquivo `mapa_grupos_descobertos.csv`.

O diretório `diagnostico_normal_only` contém o diagnóstico realizado a partir das partições efetivamente descobertas nos dados normais.

O diretório `diagnostico_oraculo` contém uma análise de referência utilizando a estrutura conhecida do gerador. Essa análise é mantida separada do procedimento principal, pois utiliza informação que não estaria disponível em um problema real.

De maneira semelhante, `probes` contém os testes realizados com os grupos descobertos, enquanto `probes_oraculo` contém os testes de referência baseados na estrutura conhecida do gerador.

A pasta `analise_direcoes` contém análises adicionais da geometria das perturbações, utilizadas especialmente para compreender os cenários B05 e B06.

2.2.4 Experimento RNN A06

Os resultados do primeiro experimento com RNN são armazenados em:

```
resultado/rnn_a06/
```

A pasta é dividida em três etapas:

```
rnn_a06/
|-- treino_monolitico/
|-- diagnostico_residuos_normal_only/
`-- teste_global_local/
```

Em `treino_monolitico` encontram-se o melhor checkpoint da RNN, o histórico de treinamento, a ordem das variáveis, os arquivos de alinhamento temporal e os resíduos normais de treinamento e validação.

Em `diagnostico_residuos_normal_only` encontram-se os diagnósticos calculados sobre esses resíduos antes da observação dos ataques.

2.2.5 Fronteira prospectiva com RNN

A pasta

`resultado/rnn_fronteira/`

contém onze subdiretórios, correspondentes aos cenários:

```
A01 A02 A03 A04 A05
B01 B02 B03 B04 B05 B06
```

Cada cenário possui a mesma organização interna:

```
CENARIO/
|-- treino_monolitico/
|-- diagnostico_residuos_normal_only/
`-- teste_global_local/
```

A pasta `treino_monolitico` contém os artefatos produzidos utilizando somente dados normais, incluindo:

- o melhor checkpoint da RNN;
- o histórico de treinamento;
- a ordem das 400 variáveis;
- os resíduos normais de treinamento;
- os resíduos normais de validação;
- os arquivos de alinhamento temporal;
- o resumo do treinamento.

A pasta `diagnostico_residuos_normal_only` contém um arquivo detalhado por grupo e um arquivo de resumo do diagnóstico estrutural realizado antes dos ataques.

Finalmente, `teste_global_local` contém dois arquivos principais: um resumo das métricas obtidas pelas estratégias global e local e um arquivo com os scores correspondentes ao teste completo.

Essa mesma estrutura é repetida para todos os onze cenários. Dessa forma, não é necessário interpretar cada pasta como um experimento diferente: elas representam execuções do mesmo protocolo sob diferentes condições estruturais.

2.3 Diretório `codigo`

O diretório `codigo` contém os programas utilizados ao longo das diferentes etapas do experimento. Os scripts foram mantidos separadamente para tornar explícitas as operações de geração de dados, auditoria, descoberta de grupos, registro de protocolos, execução dos testes e análise dos resultados.

De forma conceitual, os programas podem ser organizados nas seguintes categorias:

- geração dos conjuntos sintéticos e dos diferentes cenários;
- auditoria dos dados gerados;
- descoberta e avaliação das partições;
- experimentos exploratórios e confirmatórios;
- diagnóstico estrutural utilizando somente dados normais;
- geração e análise da fronteira estrutural;
- registro e congelamento dos protocolos experimentais;
- treinamento das RNNs;
- diagnóstico dos resíduos produzidos pelas RNNs;
- testes finais global versus local.

A geração dos dados é realizada por scripts específicos para cada etapa do experimento. Há programas destinados à construção do conjunto sintético inicial, à produção das realizações independentes utilizadas na confirmação e à geração dos diferentes cenários que compõem a fronteira estrutural.

Programas destinados à auditoria são utilizados para verificar propriedades dos dados e dos artefatos produzidos. Essas verificações incluem, entre outros aspectos, correlações, intervalos físicos, consistência das realizações independentes, características dos ataques sintéticos e propriedades dos cenários estruturais.

A descoberta das partições é realizada por scripts específicos que operam sobre os dados normais e produzem os mapas de grupos utilizados nas etapas posteriores do experimento.

Outros programas executam as comparações estáticas entre estratégias

globais e locais, incluindo os experimentos exploratórios e a avaliação de diferentes operadores de score.

A etapa confirmatória possui programas específicos para geração das realizações, congelamento das condições e execução dos testes. De forma semelhante, a fronteira estrutural possui scripts próprios para geração dos cenários, diagnóstico normal-only, análises de referência e execução dos probes.

Os programas relacionados à RNN são mantidos separadamente entre o experimento A06 e a fronteira prospectiva. Entre eles encontram-se os scripts de treinamento, diagnóstico dos resíduos e comparação final entre avaliação global e local.

O arquivo

`codigo/requirements.txt`

registra as dependências Python utilizadas pelo ambiente experimental e deve ser utilizado na preparação do ambiente de reprodução.

A descrição individual de cada programa, sua finalidade, entradas, saídas e forma de execução é apresentada na Seção 4.

A separação entre `dados`, `resultado` e `codigo` permite distinguir claramente os arquivos de entrada, os artefatos produzidos pelos experimentos e os programas responsáveis por cada etapa da execução.

Essa organização também facilita a verificação da sequência experimental, pois os protocolos e registros de integridade permanecem separados das saídas geradas posteriormente.

A ordem recomendada para reprodução dos experimentos é apresentada na Seção 3.

3 Roadmap para reprodução dos experimentos

Esta seção apresenta a ordem recomendada para reprodução dos principais experimentos. O objetivo não é repetir a descrição individual de cada programa, apresentada posteriormente na Seção 4, mas fornecer uma sequência operacional que permita reconstruir os artefatos experimentais desde a geração dos dados até a avaliação final.

Todos os comandos apresentados nesta seção pressupõem que o terminal esteja posicionado na raiz do projeto. Por exemplo:

```
cd /caminho/para/sintetico_subespacos
```

3.1 Preparação do ambiente Python

Os experimentos foram executados utilizando Python 3.10.12 em um ambiente virtual dedicado. As dependências mínimas necessárias para execução dos programas estão registradas em `codigo/requirements.txt` e podem ser instaladas por meio do comando:

```
python -m pip install -r codigo/requirements.txt
```

O arquivo `codigo/requirements.txt` especifica versões mínimas das bibliotecas e, portanto, permite a instalação de versões posteriores compatíveis. Para registrar adicionalmente o ambiente de referência efetivamente utilizado na execução dos experimentos descritos neste documento, as principais versões instaladas foram:

- Python 3.10.12;
- NumPy 2.2.6;
- pandas 2.3.3;
- SciPy 1.15.3;
- Matplotlib 3.10.9;
- scikit-learn 1.7.2;
- PyTorch 2.14.0;
- NetworkX 3.4.2;
- skrebate 0.62.

Para reproduções estritas, recomenda-se utilizar essas versões de referência ou registrar explicitamente qualquer diferença de ambiente. As restrições presentes em `codigo/requirements.txt` devem ser entendidas como requisitos mínimos de execução, e não como um congelamento exato do ambiente experimental.

3.2 Reprodução da etapa exploratória

A etapa exploratória estabelece o conjunto sintético inicial, verifica suas propriedades, obtém a primeira partição a partir dos dados normais e executa as comparações utilizadas para mapear o comportamento dos métodos antes da definição da etapa confirmatória. Uma sequência de reprodução compatível com o fluxo experimental é:

```
python codigo/gerar_normal_sintetico.py
python codigo/gerar_teste_ataque_piloto.py
python codigo/auditar_ataques_piloto.py
python codigo/auditar_correlacoes_normal.py
python codigo/auditar_ranges_fisicos.py
python codigo/auditar_plausibilidade_univariada.py
python codigo/descobrir_grupos_normal.py
python codigo/baseline_mahalanobis_global_local.py
python codigo/baselines_estaticos_global_local.py
python codigo/congelar_grade_exploratoria.py
python codigo/executar_grade_exploratoria.py
python codigo/analisar_fronteira_grade.py
```

O programa `diagnostico_mahalanobis_fpr_casado.py` constitui uma análise retrospectiva de sensibilidade e pode ser executado após `baseline_mahalanobis_global_local.py`, mas não é necessário para a calibração ou execução dos experimentos principais. De forma semelhante, o diagnóstico retrospectivo sob FPR controlado produzido por `executar_grade_exploratoria.py` pertence exclusivamente à fase exploratória e não é reutilizado para definir a etapa confirmatória posterior.

3.3 Reprodução da etapa confirmatória

A etapa confirmatória utiliza cinco realizações independentes e condições experimentais previamente congeladas. Para preservar a ordem metodológica do experimento, os eventos e as condições confirmatórias devem ser registrados antes da geração das realizações e das avaliações posteriores.

```
python codigo/congelar_eventos_confirmatorios.py
python codigo/congelar_confirmacao.py
python codigo/gerar_realizacoes_confirmatorias.py
python codigo/auditar_realizacoes_confirmatorias.py
python codigo/descobrir_grupos_confirmatorios.py
```

```
python codigo/diagnosticar_vantagem_normal_only.py
python codigo/executar_confirmacao_limpo.py
python codigo/analisar_estabilidade_confirmacao.py
python codigo/analisar_estabilidade_primaria.py
```

Nessa sequência, a descoberta das partições e o diagnóstico estrutural são realizados a partir dos dados normais antes da avaliação das perturbações. O programa `executar_confirmacao_limpo.py` corresponde à implementação consolidada utilizada para os resultados confirmatórios finais; `executar_confirmacao.py` é mantido apenas para rastreabilidade histórica e não deve ser utilizado para reproduzir essa etapa.

3.4 Preparação da fronteira estrutural

O primeiro passo consiste em registrar a grade estrutural e o protocolo que define os cenários utilizados no experimento:

```
python codigo/registrar_grade_estrutural.py
```

Essa etapa produz a definição da grade experimental, o protocolo estrutural e seu respectivo registro de integridade por SHA-256.

Em seguida, os conjuntos correspondentes aos diferentes cenários estruturais são gerados:

```
python codigo/gerar_cenarios_estruturais.py
```

Os arquivos produzidos são armazenados em `dados/fronteira_estrutural/cenarios/`, enquanto o manifesto que associa cada cenário aos respectivos conjuntos de treinamento, validação e teste é registrado em `dados/fronteira_estrutural/manifesto_cenarios_estruturais.csv`.

Após a geração, os cenários devem ser auditados:

```
python codigo/auditar_cenarios_estruturais.py
```

Essa etapa verifica a consistência dos conjuntos gerados antes que eles sejam utilizados pelos métodos de descoberta e detecção.

3.5 Descoberta dos grupos

A descoberta das partições da fronteira estrutural é realizada com:

```
python codigo/analisar_descoberta_grupos_fronteira.py
```

Para cada cenário, o programa utiliza exclusivamente as features presentes no conjunto normal de treinamento. Inicialmente é calculada a matriz de correlação absoluta de Pearson entre as 400 variáveis. Os valores das correlações entre pares são separados automaticamente em dois conjuntos por meio de KMeans, correspondentes às relações de menor e maior intensidade. A partir dessa separação é definido o limiar de conectividade utilizado para construir o grafo, cujas componentes conexas constituem os grupos descobertos.

O mapa de referência utilizado pelo gerador sintético não participa dessa etapa de descoberta. Para cada cenário, a partição obtida é primeiramente salva em:

```
resultado/fronteira_estrutural/descoberta_grupos/CENARIO/
    mapa_grupos_descobertos.csv
```

Somente depois da construção e gravação dessa partição o mapa de referência é consultado para calcular medidas de qualidade da descoberta, como ARI e NMI.

Essa separação é importante porque distingue duas questões diferentes: descobrir uma possível estrutura de subespaços a partir dos dados normais e avaliar posteriormente quanto essa estrutura coincide com a organização conhecida do gerador sintético.

3.6 Diagnóstico estrutural com dados normais

Após a descoberta dos grupos, o diagnóstico estrutural principal é executado com:

```
python codigo/diagnosticar_fronteira_normal_only.py
```

Essa etapa utiliza somente informações provenientes dos conjuntos normais de treinamento e validação. Nenhum ataque é utilizado para calcular o diagnóstico.

As saídas principais são armazenadas em:

```
resultado/fronteira_estrutural/diagnostico_normal_only/
```

e incluem resultados por grupo e por cenário.

Somente depois desse diagnóstico são executados os probes contendo perturbações sintéticas:

```
python codigo/executar_probes_fronteira_estrutural.py
```

Os resultados são armazenados em:

```
resultado/fronteira_estrutural/probes/
```

Dessa forma, o fluxo principal da fronteira estrutural pode ser resumido por:

$$\text{geração} \rightarrow \text{auditoria} \rightarrow \text{descoberta} \rightarrow \text{diagnóstico normal-only} \rightarrow \text{probes}. \tag{18}$$

3.7 Análises de referência

O repositório também contém análises que utilizam diretamente a estrutura de referência conhecida do gerador sintético. Elas não fazem parte do procedimento que seria aplicado a um conjunto de dados real e são utilizadas apenas como instrumentos de análise.

Entre elas estão o diagnóstico baseado na partição de referência e a execução dos probes utilizando essa mesma partição:

```
python codigo/diagnosticar_fronteira_oraculo.py
python codigo/executar_probes_fronteira_oraculo.py
```

Também foi realizada uma análise posterior das direções das perturbações nos cenários B05 e B06:

```
python codigo/analisar_direcoes_b05_b06.py
```

Esses programas devem ser interpretados como análises explicativas e não como componentes necessários para a aplicação do diagnóstico normal-only.

3.8 Reprodução do experimento inicial com RNN no cenário A06

O cenário A06 foi utilizado como experimento com RNN anterior à definição da bateria prospectiva e possui protocolo e artefatos próprios. Para repro-

duzir essa execução preservando sua sequência metodológica, devem ser executados, nesta ordem:

```
python codigo/codigo/registrar_protocolo_rnn_a06.py
python codigo/treinar_rnn_a06_residuos_normais.py
python codigo/diagnosticar_rnn_a06_residuos_normal_only.py
python codigo/testar_rnn_a06_global_local.py
```

Essa execução utiliza o mesmo princípio posteriormente adotado na bateria prospectiva: uma única RNN monolítica produz os resíduos utilizados pelas estratégias global e local. Entretanto, por ter sido realizada antes da definição e do congelamento da bateria prospectiva, A06 deve permanecer identificado como experimento anterior de referência e não como integrante dos onze cenários prospectivos.

3.9 Registro do protocolo da fronteira com RNN

Antes do treinamento das RNNs da etapa prospectiva, o protocolo experimental é registrado com:

```
python codigo/registrar_protocolo_rnn_fronteira.py
```

O protocolo e seu hash SHA-256 são armazenados em:

```
dados/rnn_fronteira/
```

A bateria prospectiva utiliza onze cenários:

```
A01 A02 A03 A04 A05
B01 B02 B03 B04 B05 B06
```

O cenário A06 não pertence a essa bateria prospectiva, pois já havia sido utilizado anteriormente como experimento inicial com RNN. Seus artefatos são mantidos separadamente em `dados/rnn_a06/` e `resultado/rnn_a06/`.

3.10 Treinamento das RNNs

Para cada cenário prospectivo é treinada uma única RNN monolítica. O cenário é informado como argumento do programa.

Por exemplo, para A01:

```
python codigo/treinar_rnn_fronteira_residuos_normais.py A01
```

Para reproduzir outro cenário, basta substituir `A01` pelo identificador desejado.

Por exemplo:

```
python codigo/treinar_rnn_fronteira_residuos_normais.py B03
```

O treinamento utiliza somente os conjuntos normais de treinamento e validação. Entre os artefatos produzidos estão o melhor checkpoint da RNN, o histórico de treinamento, a ordem das features e os resíduos normais de treinamento e validação.

3.11 Diagnóstico dos resíduos da RNN

Após o treinamento de cada cenário, o diagnóstico normal-only é aplicado aos resíduos produzidos pela mesma RNN:

```
python
↪ codigo/diagnosticar_rnn_fronteira_residuos_normal_only.py
↪ A01
```

Novamente, o identificador final deve ser substituído pelo cenário que está sendo processado.

Essa etapa utiliza exclusivamente os resíduos normais de treinamento e validação. Os ataques ainda não são utilizados.

Os diagnósticos produzidos para os onze cenários foram posteriormente registrados em manifesto SHA-256, juntamente com os artefatos de treinamento, antes da abertura da etapa de teste.

3.12 Registro de integridade antes dos testes

Depois que o treinamento e o diagnóstico normal-only foram concluídos para todos os cenários prospectivos, os artefatos correspondentes foram registrados em manifestos SHA-256 antes da abertura da etapa contendo ataques. O manifesto dos artefatos de treinamento é produzido com:

```
find resultado/rnn_fronteira \
  \( -name 'modelo_rnn_*_best.pth' -o \
     -name 'residuos_treino.npy' -o \
     -name 'residuos_validacao.npy' -o \
     -name 'ordem_features.csv' \) \
```

```
  -type f -print0 | sort -z | xargs -0 sha256sum \
  > dados/rnn_fronteira/sha256_artefatos_treino.txt
```

O manifesto dos diagnósticos produzidos exclusivamente com dados normais é gerado com:

```
find resultado/rnn_fronteira \
  -path
  ↪  '*/diagnostico_residuos_normal_only/diagnostico_residuos_resumo.csv'
  ↪  \
  -type f -print0 | sort -z | xargs -0 sha256sum \
  > dados/rnn_fronteira/sha256_diagnosticos_normal_only.txt
```

Esses dois manifestos devem existir antes da execução de `testar_rnn_fronteira_global_local.py`, que os utiliza para verificar a integridade dos artefatos produzidos anteriormente.

3.13 Teste global versus local

Somente depois da conclusão dos treinamentos, dos diagnósticos normal-only e do registro de integridade desses artefatos é realizada a avaliação contendo os ataques.

Para um cenário específico, o comando é:

```
python codigo/testar_rnn_fronteira_global_local.py A01
```

O mesmo padrão é utilizado para os demais cenários.

Esse programa utiliza o mesmo checkpoint e o mesmo vetor de resíduos para as duas estratégias avaliadas. A diferença entre elas ocorre somente na forma como esses resíduos são utilizados para produzir o score de anomalia global ou os scores locais.

Os resultados de cada cenário são armazenados em:

```
resultado/rnn_fronteira/CENARIO/teste_global_local/
```

Após a conclusão dos testes para todos os cenários prospectivos, os arquivos finais de scores e resultados são registrados em um manifesto SHA-256 por meio do comando:

```
find resultado/rnn_fronteira \
  \( -name 'resultados_rnn_*_global_local.csv' -o \
     -name 'scores_rnn_*_global_local.csv' \) \
```

```
-type f -print0 | sort -z | xargs -0 sha256sum \
> dados/rnn_fronteira/sha256_resultados_testes.txt
```

Esse manifesto é produzido somente após a avaliação com ataques e tem a função de preservar o estado dos resultados obtidos, não participando da calibração dos detectores nem das decisões anteriores ao teste.

Assim, a sequência principal da etapa prospectiva com RNN é:

$$\text{protocolo} \rightarrow \text{treinamento normal} \rightarrow \text{resíduos normais} \rightarrow \text{diagnóstico normal-only} \rightarrow \text{registro c} \tag{19}$$

4 Programas e scripts

Esta seção descreve individualmente os programas utilizados na construção, auditoria e execução dos experimentos. Para cada script são apresentados sua finalidade principal, sua posição no fluxo experimental e, quando aplicável, a forma de execução. Os programas são descritos aproximadamente na ordem em que aparecem no processo experimental, e não em ordem alfabética.

Os comandos apresentados pressupõem que o terminal esteja posicionado na raiz do projeto `sintetico_subespacos/` e que o ambiente Python contendo as dependências especificadas em `codigo/requirements.txt` esteja ativo.

Os scripts que não recebem argumentos são executados diretamente com o interpretador Python. Nos programas da fronteira com RNN que exigem a identificação de um cenário, o argumento `CENARIO` deve ser substituído por um dos identificadores permitidos, como `A01`, `A02` ou `B06`.

4.1 `gerar_normal_sintetico.py`

O programa `gerar_normal_sintetico.py` implementa o gerador sintético utilizado como base dos experimentos iniciais. O sistema é composto por 20 grupos, cada um contendo 14 variáveis contínuas e 6 variáveis binárias, totalizando 400 variáveis observadas. Para as variáveis contínuas, a dependência estatística é produzida pela combinação de um fator latente es-

pecífico do grupo, um fator global compartilhado e ruído individual, com parâmetros iniciais α = 0,65 para a contribuição intragrupo e β = 0,05 para a contribuição global. O gerador também introduz dependência temporal nos fatores latentes e cria parâmetros físicos fixos para os sensores, reutilizados entre os diferentes conjuntos de dados para impedir que treinamento e validação tenham escalas físicas distintas. Na configuração piloto são geradas 20.000 amostras para treinamento e 10.000 para validação. O script utiliza sementes determinísticas para permitir a reprodução tanto dos dados quanto dos parâmetros físicos dos sensores.

```
python codigo/gerar_normal_sintetico.py
```

4.2 gerar_teste_ataque_piloto.py

O programa `gerar_teste_ataque_piloto.py` constrói o conjunto de teste utilizado no experimento piloto. O script utiliza os parâmetros físicos dos sensores e o mapa de referência dos grupos, gera um conjunto normal de teste com 20.000 amostras e cria, a partir dele, uma versão contendo perturbações sintéticas. Na configuração piloto, dez variáveis são modificadas em cada evento de ataque. Sementes independentes são utilizadas para a geração temporal do teste e para a definição da ordem das variáveis, permitindo reproduzir exatamente o mesmo conjunto e a mesma seleção de atributos. Como saída, o programa preserva tanto o conjunto normal de base quanto a versão contendo ataques e grava separadamente um arquivo que descreve os eventos introduzidos.

```
python codigo/gerar_teste_ataque_piloto.py
```

4.3 auditar_ataques_piloto.py

O programa `auditar_ataques_piloto.py` verifica a consistência dos ataques inseridos no experimento piloto. Para isso, compara o conjunto normal de teste com sua versão perturbada, utiliza o arquivo que descreve os eventos e consulta o mapa de referência das variáveis. A auditoria permite identificar quantas variáveis foram efetivamente alteradas em cada evento e verificar se as modificações introduzidas correspondem à estrutura registrada para o experimento. Os resultados da verificação são gravados em diretório próprio dentro de `resultado/auditoria_ataques_piloto/`.

```
python codigo/auditar_ataques_piloto.py
```

4.4 auditar_correlacoes_normal.py

O programa `auditar_correlacoes_normal.py` examina a estrutura de correlação presente no conjunto normal de treinamento do experimento piloto. O script utiliza o mapa de referência apenas para organizar e interpretar posteriormente as relações observadas entre as variáveis, produzindo arquivos com as correlações entre pares de atributos e resumos agregados. As saídas são armazenadas em `resultado/auditoria_correlacoes/` e permitem verificar se a estrutura estatística efetivamente produzida pelo gerador apresenta o comportamento esperado antes da aplicação dos métodos de descoberta de grupos.

```
python codigo/auditar_correlacoes_normal.py
```

4.5 auditar_ranges_fisicos.py

O programa `auditar_ranges_fisicos.py` verifica se as variáveis contínuas dos conjuntos normais de treinamento e validação respeitam os intervalos físicos definidos pelo gerador. Para cada variável, o script utiliza os parâmetros físicos previamente registrados e contabiliza eventuais observações abaixo ou acima dos limites previstos. O mapa de referência também é carregado para permitir a organização das verificações por variável e grupo. O resultado da auditoria é gravado em `resultado/auditoria_ranges/` e serve como controle de consistência dos dados antes das etapas posteriores do experimento.

```
python codigo/auditar_ranges_fisicos.py
```

4.6 descobrir_grupos_normal.py

O programa `descobrir_grupos_normal.py` realiza a descoberta inicial dos subespaços a partir do conjunto normal de treinamento. O procedimento calcula a matriz de correlação de Pearson entre as variáveis, considera os valores absolutos das correlações e utiliza KMeans para separar automaticamente as relações em dois conjuntos interpretados como correlações fracas e fortes. A partir dessa separação é definido o critério utilizado para construir o grafo de relações entre as variáveis e identificar seus componentes, que

passam a representar os grupos descobertos. O programa grava o mapa das variáveis para os grupos obtidos, informações sobre os componentes encontrados e arquivos de avaliação da partição, permitindo comparar posteriormente a estrutura descoberta com a organização de referência conhecida do gerador sintético.

```
python codigo/descobrir_grupos_normal.py
```

4.7 baseline_mahalanobis_global_local.py

O programa `baseline_mahalanobis_global_local.py` executa a primeira comparação direta entre detecção global e detecção local utilizando o score quadrático de Mahalanobis. O estimador de covariância utilizado é Ledoit–Wolf. Na estratégia global, uma única distribuição é ajustada considerando conjuntamente todas as variáveis; na estratégia local, modelos equivalentes são ajustados separadamente para os grupos previamente descobertos. Os limiares são calibrados no conjunto normal de validação para uma taxa nominal de falsos positivos de 1%. O programa utiliza os conjuntos de treinamento, validação e teste, os parâmetros físicos, o mapa de grupos e a descrição dos eventos de ataque, e produz scores, métricas e informações por evento para permitir a comparação das duas estratégias sob o mesmo protocolo experimental.

```
python codigo/baseline_mahalanobis_global_local.py
```

4.8 baselines_estaticos_global_local.py

O programa `baselines_estaticos_global_local.py` amplia a comparação anterior para diferentes operadores estáticos de score, permitindo verificar se uma eventual vantagem da decomposição depende do modo como a evidência das diferentes variáveis é agregada. O script avalia Energia, MAX, TOP3, TOP5 e TOP10 e também constrói as correspondentes avaliações locais, nas quais os scores são calculados dentro dos grupos e posteriormente combinados. Nesse experimento, Energia corresponde à média dos desvios padronizados elevados ao quadrado e não deve ser confundida com o erro quadrático médio de reconstrução utilizado posteriormente no treinamento da RNN. Os limiares são definidos utilizando exclusivamente o conjunto normal de validação, novamente com FPR nominal de 1%, e as métricas são calculadas posteriormente no conjunto de teste. Essa etapa funciona

como controle para distinguir efeitos causados pela decomposição daqueles decorrentes especificamente do operador de score utilizado.

```
python codigo/baselines_estaticos_global_local.py
```

4.9 diagnostico_mahalanobis_fpr_casado.py

O programa `diagnostico_mahalanobis_fpr_casado.py` realiza uma análise auxiliar posterior sobre os scores já produzidos pelo experimento de Mahalanobis. Seu objetivo é comparar as estratégias global e local sob um FPR efetivamente casado no próprio conjunto de teste, permitindo investigar se diferenças observadas entre os métodos poderiam ser explicadas apenas por diferenças no número de falsos positivos.

Nota metodológica: por utilizar informações do conjunto de teste para realizar esse pareamento, esta análise é exclusivamente retrospectiva e de sensibilidade. Ela não participa da calibração dos detectores, da definição dos limiares, da seleção dos modelos ou dos resultados principais utilizados para avaliar o método. Nos experimentos principais, os limiares são definidos previamente utilizando exclusivamente dados normais de validação.

```
python codigo/diagnostico_mahalanobis_fpr_casado.py
```

4.10 congelar_grade_exploratoria.py

O programa `congelar_grade_exploratoria.py` registra previamente a grade utilizada na etapa exploratória e o protocolo correspondente. O script define as combinações de parâmetros que serão avaliadas, incluindo diferentes intensidades de perturbação, e fixa o FPR nominal de validação em 1%. Como saída, são produzidos o arquivo `dados/grade_exploratoria.csv` e o protocolo `dados/protocolo_grade_exploratoria.json`. O próprio protocolo também registra que eventual casamento de FPR realizado posteriormente no conjunto de teste possui finalidade exclusivamente diagnóstica e não deve ser confundido com o procedimento de calibração empregado nos experimentos principais.

```
python codigo/congelar_grade_exploratoria.py
```

4.11 executar_grade_exploratoria.py

O programa executar_grade_exploratoria.py executa a bateria exploratória de perturbações sobre o conjunto sintético inicial. O script utiliza os conjuntos normais de treinamento, validação e teste de base, a grade previamente registrada, os eventos congelados, os parâmetros físicos e os grupos descobertos. Os métodos são avaliados sob limiares definidos a partir da validação normal com FPR nominal de 1%. Além das métricas obtidas com esses limiares, o programa calcula separadamente um diagnóstico retrospectivo no conjunto de teste sob FPR limitado a 1%, utilizado somente para mapear regiões em que a detecção local apresenta maior potência. Os resultados são armazenados em resultado/grade_exploratoria/, incluindo os resultados por método, os resumos por condição e a relação das condições nas quais alguma estratégia local supera as estratégias globais analisadas.

Nota metodológica: as medidas identificadas nos arquivos como diagnóstico sob FPR controlado utilizam os rótulos do conjunto de teste e, portanto, pertencem exclusivamente à fase exploratória. Elas não são utilizadas para definir os limiares, escolher modelos ou sustentar sozinhas a avaliação confirmatória posterior. O próprio programa registra explicitamente essa distinção.

```
python codigo/executar_grade_exploratoria.py
```

4.12 congelar_eventos_confirmatorios.py

O programa congelar_eventos_confirmatorios.py registra previamente os eventos que serão utilizados na etapa confirmatória e a ordem das variáveis elegíveis para receber as perturbações. Uma semente fixa é utilizada para tornar reproduzível essa ordenação. O script grava os eventos em dados/eventos_confirmatorios_base.csv, preserva separadamente a ordem das features e calcula hashes SHA-256 desses arquivos. O objetivo é garantir que posição, duração, direção e seleção potencial das variáveis dos ataques permaneçam fixadas antes da execução dos experimentos confirmatórios.

```
python codigo/congelar_eventos_confirmatorios.py
```

4.13 congelar_confirmacao.py

O programa congelar_confirmacao.py define e registra as condições utilizadas na etapa confirmatória. O script fixa previamente as intensidades das perturbações, as sementes empregadas nas diferentes realizações independentes e o FPR nominal de validação de 1%. Como saída são produzidos dados/condicoes_confirmatorias.csv e dados/protocolo_confirmatorio.json. O protocolo declara explicitamente que não será realizado casamento de FPR utilizando o conjunto de teste, estabelecendo antes da execução a separação entre calibração por validação normal e avaliação final.

```
python codigo/congelar_confirmacao.py
```

4.14 gerar_realizacoes_confirmatorias.py

O programa gerar_realizacoes_confirmatorias.py produz as realizações independentes utilizadas na etapa confirmatória. Antes da geração, o script calcula e verifica hashes dos arquivos que definem as condições e o protocolo, impedindo que a execução prossiga caso esses artefatos tenham sido modificados em relação ao estado previamente registrado. Para cada realização são produzidos conjuntos separados de treinamento normal, validação normal e teste normal de base. Os arquivos são organizados em subdiretórios próprios e um manifesto final registra a associação entre cada realização e seus respectivos conjuntos de dados.

```
python codigo/gerar_realizacoes_confirmatorias.py
```

4.15 descobrir_grupos_confirmatorios.py

O programa descobrir_grupos_confirmatorios.py aplica o procedimento de descoberta de grupos separadamente a cada uma das realizações independentes utilizadas na confirmação. O script lê o manifesto das realizações, constrói uma partição a partir dos respectivos dados normais e grava o mapa de grupos descobertos para cada realização em resultado/grupos_confirmatorios/. Depois de obtida a partição, o mapa de referência do gerador é utilizado para avaliar a correspondência entre a estrutura descoberta e a estrutura conhecida, sendo também produzido um resumo consolidado das diferentes realizações. Dessa forma, a qualidade da descoberta pode

ser avaliada sem que a informação de referência seja utilizada para formar previamente os grupos.

```
python codigo/descobrir_grupos_confirmatorios.py
```

4.16 auditar_realizacoes_confirmatorias.py

O programa `auditar_realizacoes_confirmatorias.py` verifica a consistência das realizações independentes produzidas para a etapa confirmatória. O script percorre o manifesto das realizações e confronta os conjuntos gerados com os parâmetros físicos e com a estrutura de referência do gerador, produzindo um conjunto consolidado de verificações em diretório próprio de auditoria. Essa etapa funciona como controle anterior à execução da confirmação, permitindo detectar inconsistências nos dados independentes antes que eles sejam utilizados nas comparações entre os métodos.

```
python codigo/auditar_realizacoes_confirmatorias.py
```

4.17 analisar_estabilidade_confirmacao.py

O programa `analisar_estabilidade_confirmacao.py` realiza uma análise agregada da estabilidade dos resultados obtidos nas diferentes realizações da etapa confirmatória. A partir do arquivo consolidado produzido pela execução da confirmação, o script reúne métricas das estratégias global e local, incluindo suas taxas de falsos positivos, e calcula estatísticas resumidas entre as realizações. O resultado é gravado em um arquivo específico de análise de estabilidade, permitindo verificar se o comportamento observado permanece consistente quando o experimento é repetido com diferentes realizações independentes dos dados.

```
python codigo/analisar_estabilidade_confirmacao.py
```

4.18 analisar_estabilidade_primaria.py

Na comparação primária resumida pelo programa, o denominado *melhor global* não representa um detector global previamente selecionado. Ele é definido retrospectivamente, para cada par realização–condição, como o operador global que obteve o maior recall no conjunto de teste. Essa seleção é utilizada apenas como benchmark conservador para a comparação

com `Local_Mahalanobis` e não altera os limiares ou qualquer componente calibrado previamente com dados normais.

```
python codigo/analisar_estabilidade_primaria.py
```

4.19 diagnosticar_vantagem_normal_only.py

O programa `diagnosticar_vantagem_normal_only.py` implementa o diagnóstico estrutural construído exclusivamente a partir dos dados normais das realizações confirmatórias. Para cada realização, o script utiliza o manifesto dos conjuntos de dados, os parâmetros previamente registrados e os grupos descobertos, mantendo FPR nominal de 1% para a definição dos limiares de validação. O programa produz resultados tanto no nível dos grupos quanto no nível consolidado das realizações, armazenados em `resultado/diagnostico_normal_only/`. Essa etapa é metodologicamente distinta da avaliação com ataques: seu objetivo é estimar, antes da introdução de anomalias, se a estrutura estatística observada oferece condições favoráveis para que a avaliação local preserve evidência que poderia ser atenuada pela avaliação global.

```
python codigo/diagnosticar_vantagem_normal_only.py
```

4.20 executar_confirmacao_limpo.py

O programa `executar_confirmacao_limpo.py` constitui a versão consolidada da execução confirmatória. Antes de iniciar os experimentos, o script verifica os hashes dos arquivos previamente congelados, incluindo condições, protocolo, eventos e ordem das variáveis, interrompendo a execução caso algum desses artefatos tenha sido alterado. O programa processa cinco realizações independentes e seis condições confirmatórias, verifica a presença das 400 features e utiliza, em cada realização, a partição produzida previamente pelo procedimento de descoberta de grupos. A quantidade e a composição desses grupos são determinadas pelo próprio procedimento de agrupamento aplicado aos dados normais de cada realização. Em seguida, são executadas comparações globais e locais utilizando operadores estáticos e o score quadrático de Mahalanobis com estimação de covariância por Ledoit–Wolf. Os limiares são determinados exclusivamente a partir dos conjuntos normais de validação, com FPR nominal previamente definido, e permanecem fixos durante a avaliação do teste. A estratégia `Local_Mahalanobis` é

registrada como método local primário para a comparação confirmatória, enquanto os demais operadores funcionam como comparações adicionais e controles metodológicos. O script também realiza verificações internas, como a equivalência esperada entre `Global_MAX` e `Local_MAX`, e produz resultados detalhados, comparação primária e resumos agregados das diferentes realizações.

Na comparação primária produzida por essa rotina, o denominado *melhor global* é definido retrospectivamente, para cada par realização–condição, como o operador global que apresenta o maior recall no conjunto de teste. Essa seleção é utilizada exclusivamente como benchmark conservador para a comparação com `Local_Mahalanobis`; ela não participa da calibração dos limiares, da descoberta da partição ou do ajuste dos detectores, que permanecem definidos anteriormente a partir dos dados normais.

```
python codigo/executar_confirmacao_limpo.py
```

4.21 `executar_confirmacao.py`

O programa `executar_confirmacao.py` corresponde a uma versão anterior da rotina de execução confirmatória, preservada no repositório para rastreabilidade do desenvolvimento experimental. Ele já implementava a verificação dos artefatos congelados, a utilização das realizações independentes, a calibração dos limiares pela validação normal e a comparação entre estratégias globais e locais. Durante a consolidação do experimento, essa rotina foi posteriormente ampliada e reorganizada em `executar_confirmacao_limpo.py`, que passou a incluir verificações adicionais, operadores locais estáticos e uma separação explícita da comparação primária. Portanto, este script histórico não é necessário para reproduzir a versão final dos resultados confirmatórios apresentada no repositório.

Observação de reprodutibilidade: para reproduzir os resultados confirmatórios finais deve ser utilizado `executar_confirmacao_limpo.py`. O arquivo `executar_confirmacao.py` é mantido somente como registro de uma etapa anterior do desenvolvimento do experimento.

4.22 registrar_grade_estrutural.py

O programa registrar_grade_estrutural.py define e registra a grade de cenários utilizada para estudar a fronteira estrutural do problema. Cada cenário especifica uma combinação controlada dos parâmetros que determinam a dependência intragrupo e a dependência compartilhada entre grupos, permitindo variar gradualmente a facilidade com que uma estrutura de subespaços pode ser identificada nos dados normais. O script grava a grade experimental e o protocolo correspondente em dados/fronteira_estrutural/ e calcula hashes SHA-256 desses artefatos, criando um registro de integridade anterior à geração dos conjuntos de dados.

```
python codigo/registrar_grade_estrutural.py
```

4.23 gerar_cenarios_estruturais.py

O programa gerar_cenarios_estruturais.py produz os conjuntos de dados correspondentes aos cenários definidos na grade estrutural. Antes da geração, o script verifica os arquivos previamente registrados e utiliza os parâmetros físicos e o mapa de referência do gerador. Para cada cenário são criados conjuntos separados de treinamento, validação e teste, utilizando sementes fixas distintas para cada divisão, o que permite reproduzir exatamente as mesmas séries temporais. Os arquivos resultantes são armazenados em dados/fronteira_estrutural/cenarios/, e um manifesto registra os caminhos e os parâmetros associados a cada cenário.

```
python codigo/gerar_cenarios_estruturais.py
```

4.24 auditar_cenarios_estruturais.py

O programa auditar_cenarios_estruturais.py verifica a consistência dos conjuntos produzidos para a fronteira estrutural antes que eles sejam utilizados nas etapas de descoberta e diagnóstico. O script percorre o manifesto dos cenários, consulta os parâmetros físicos e o mapa de referência do gerador e produz um arquivo consolidado com os resultados das verificações. Essa auditoria funciona como controle de integridade dos cenários e permite identificar problemas de geração, estrutura ou faixa das variáveis antes da execução dos experimentos subsequentes.

```
python codigo/auditar_cenarios_estruturais.py
```

4.25 analisar_descoberta_grupos_fronteira.py

O programa analisar_descoberta_grupos_fronteira.py executa a descoberta dos grupos separadamente para cada cenário da fronteira estrutural. O procedimento utiliza somente as variáveis observadas no conjunto normal de treinamento, calcula as correlações absolutas de Pearson entre os pares de features e utiliza KMeans para separar automaticamente relações de menor e maior intensidade. A partir dessa separação é construído o grafo utilizado para identificar os componentes que formam a partição descoberta. O mapa obtido é salvo antes de qualquer comparação com a estrutura conhecida do gerador. Somente após essa etapa o mapa de referência é carregado para calcular medidas de concordância e outras estatísticas destinadas a avaliar a qualidade da descoberta. Dessa forma, a estrutura de referência é utilizada como instrumento de avaliação e não como informação fornecida ao algoritmo de agrupamento.

```
python codigo/analisar_descoberta_grupos_fronteira.py
```

4.26 diagnosticar_fronteira_normal_only.py

O programa diagnosticar_fronteira_normal_only.py aplica o diagnóstico estrutural aos cenários da fronteira utilizando exclusivamente dados normais e as partições previamente descobertas. Para cada cenário são estimados modelos globais e locais baseados no score quadrático de Mahalanobis, com covariância regularizada por Ledoit–Wolf, sendo os limiares determinados a partir da validação normal com FPR nominal de 1%. A partir das distribuições globais e locais o programa calcula as quantidades utilizadas pelo diagnóstico de vantagem estrutural e produz resultados tanto no nível dos grupos quanto no nível consolidado dos cenários. Nenhuma informação de ataque é utilizada nessa etapa. As saídas são gravadas em resultado/fronteira_estrutural/diagnostico_normal_only/.

```
python codigo/diagnosticar_fronteira_normal_only.py
```

4.27 diagnosticar_fronteira_oraculo.py

O programa diagnosticar_fronteira_oraculo.py executa uma análise paralela ao diagnóstico normal-only, substituindo a partição descoberta pela partição de referência conhecida do gerador sintético. Os mesmos princípios

de estimação global e local, regularização por Ledoit–Wolf e calibração dos limiares pela validação normal são mantidos. O objetivo dessa rotina não é representar um procedimento disponível em aplicação real, mas separar dois fatores experimentais distintos: a utilidade potencial de uma determinada partição para a detecção de anomalias e a capacidade do algoritmo de descoberta de reconstruir essa partição a partir dos dados normais.

Nota metodológica: os resultados produzidos por este programa devem ser interpretados como análise de referência. A partição utilizada é conhecida porque o conjunto é sintético e, portanto, não pode ser tratada como informação disponível ao método em uma aplicação real. Além disso, a partição de referência não deve ser interpretada automaticamente como uma estrutura estatisticamente recuperável a partir dos dados normais em todos os cenários. Sua função nessa análise é fornecer uma organização controlada definida pelo próprio mecanismo gerador, permitindo avaliar separadamente a utilidade potencial da decomposição e a capacidade do procedimento de descoberta de recuperar essa estrutura.

```
python codigo/diagnosticar_fronteira_oraculo.py
```

4.28 executar_probes_fronteira_estrutural.py

O programa `executar_probes_fronteira_estrutural.py` executa as perturbações sintéticas utilizadas para avaliar empiricamente o comportamento associado às condições identificadas pelo diagnóstico normal-only ao longo da fronteira estrutural. O script utiliza a grade e o protocolo previamente registrados, o manifesto dos cenários, os eventos e a ordem das variáveis previamente definidos, os parâmetros físicos e, para cada cenário, a partição obtida pelo procedimento de descoberta de grupos. Os limiares global e local são calibrados a partir dos dados normais com FPR nominal de 1% e permanecem fixos durante a introdução das perturbações. Como saída, o programa produz resultados globais e locais, resultados no nível dos eventos e um resumo agregado por cenário. Dessa forma, os probes são utilizados como instrumento experimental para verificar se as condições estruturais identificadas anteriormente se refletem em diferenças observáveis de detecção, sem utilizar os ataques para construir a partição ou calcular previamente o diagnóstico estrutural.

```
python codigo/executar_probes_fronteira_estrutural.py
```

4.29 executar_probes_fronteira_oraculo.py

O programa executar_probes_fronteira_oraculo.py repete a avaliação dos probes da fronteira estrutural substituindo os grupos descobertos pela partição de referência conhecida do gerador sintético. O restante do protocolo permanece equivalente, incluindo a utilização da mesma grade estrutural, dos mesmos eventos, da mesma ordem de variáveis, dos mesmos parâmetros físicos e da mesma regra de calibração dos limiares com FPR nominal de 1%. Essa execução permite distinguir situações em que uma decomposição possui utilidade para a detecção, mas a estrutura correspondente não foi recuperada adequadamente pelo procedimento de descoberta, de situações em que a própria decomposição não oferece vantagem relevante sob as condições analisadas.

Nota metodológica: esta rotina é uma análise de referência e não representa um procedimento operacional aplicável a dados reais, pois utiliza deliberadamente uma informação estrutural conhecida apenas porque o conjunto de dados foi gerado sinteticamente.

A finalidade dessa comparação é avaliar a utilidade potencial da decomposição quando a referência utilizada para construir o experimento é conhecida. Ela não pressupõe que essa mesma partição possa ser inferida dos dados normais em todos os pontos da fronteira estrutural.

```
python codigo/executar_probes_fronteira_oraculo.py
```

4.30 analisar_direcoes_b05_b06.py

O programa analisar_direcoes_b05_b06.py realiza uma análise posterior específica dos cenários B05 e B06, nos quais o diagnóstico conservador baseado na pior direção não fornece uma garantia de vantagem local válida para todas as direções do subespaço, embora determinadas perturbações ainda possam favorecer a estratégia local. O script utiliza os dados dos cenários, os eventos previamente definidos, a ordem das variáveis, os parâmetros físicos, a partição de referência e os resultados do diagnóstico estrutural para examinar a relação entre a direção efetivamente aplicada pela perturbação e a geometria estatística local. São produzidos arquivos detalhados e um resumo consolidado dessa análise.

Nota metodológica: esta análise é explicativa e posterior aos resultados

principais. Seu objetivo é interpretar por que a ausência de uma garantia conservadora para todas as direções não implica necessariamente ausência de vantagem para uma direção específica de anomalia. Portanto, ela não é utilizada para selecionar cenários, ajustar limiares ou definir previamente os ataques empregados nas avaliações principais.

```
python codigo/analisar_direcoes_b05_b06.py
```

4.31 registrar_protocolo_rnn_fronteira.py

O programa `registrar_protocolo_rnn_fronteira.py` registra o protocolo utilizado na avaliação prospectiva da fronteira estrutural com RNN. O script reúne informações provenientes do manifesto dos cenários, dos eventos previamente definidos, da ordem das variáveis atacadas, dos parâmetros físicos e da implementação base da RNN, produzindo um arquivo de protocolo e seu respectivo hash SHA-256 em `dados/rnn_fronteira/`. O protocolo registra previamente as estratégias global e local baseadas no score quadrático de Mahalanobis com estimação de covariância Ledoit–Wolf, bem como a calibração dos limiares para FPR nominal de 1% utilizando a mesma validação normal. Esse registro é realizado antes da etapa final de teste e funciona como referência fixa para as execuções posteriores.

```
python codigo/registrar_protocolo_rnn_fronteira.py
```

4.32 codigo/codigo/registrar_protocolo_rnn_a06.py

O programa `codigo/codigo/registrar_protocolo_rnn_a06.py` registra o protocolo específico utilizado no experimento inicial com RNN do cenário A06. O script reúne o manifesto do cenário estrutural, os eventos previamente definidos, a ordem das variáveis atacadas, os parâmetros físicos, a partição descoberta para A06 e a implementação base da RNN, registrando esses elementos antes das etapas de treinamento, diagnóstico e teste. O protocolo especifica as estratégias global e local baseadas no score quadrático de Mahalanobis com estimação de covariância Ledoit–Wolf e estabelece a calibração independente dos limiares para FPR nominal de 1% utilizando dados normais de validação. Como saída são produzidos `dados/rnn_a06/protocolo_rnn_a06.json` e o respectivo registro SHA-256 `dados/rnn_a06/sha256_protocolo_rnn_a06.txt`, permitindo verificar posteriormente que o experimento foi executado sob a configuração

previamente registrada.

```
python codigo/codigo/registrar_protocolo_rnn_a06.py
```

Observação sobre a organização do repositório: nesta versão do projeto, este programa encontra-se no subdiretório `codigo/codigo/`. Embora essa localização decorra da organização interna do projeto, o caminho é preservado neste documento para corresponder exatamente à estrutura utilizada nas execuções registradas.

4.33 `treinar_rnn_a06_residuos_normais.py`

O programa `treinar_rnn_a06_residuos_normais.py` realiza o treinamento da RNN utilizada no experimento inicial do cenário A06. Essa execução antecede a bateria prospectiva da fronteira com RNN e, por esse motivo, A06 deve ser interpretado como experimento prévio de referência, e não como integrante do conjunto prospectivo posteriormente congelado. O script verifica a integridade do protocolo correspondente antes do treinamento, utiliza somente dados normais e adota janela temporal de 60 amostras, batch de 512, camada recorrente com 64 unidades ocultas, máximo de 100 épocas, early stopping com paciência de 5 épocas e semente 42. Ao final, são preservados o melhor checkpoint, o histórico de treinamento, a ordem das features, os resíduos de treinamento e validação, os arquivos de alinhamento temporal e um resumo da execução.

```
python codigo/treinar_rnn_a06_residuos_normais.py
```

4.34 `diagnosticar_rnn_a06_residuos_normal_only.py`

O programa `diagnosticar_rnn_a06_residuos_normal_only.py` aplica o diagnóstico estrutural aos resíduos normais produzidos pela RNN do cenário A06. Em vez de operar diretamente sobre as variáveis originais, essa etapa utiliza os vetores de resíduos de treinamento e validação, preservando a representação efetivamente utilizada pelo detector sequencial. São estimados modelos globais e locais de Mahalanobis com covariância Ledoit–Wolf, e os limiares são calculados exclusivamente a partir da validação normal com FPR nominal de 1%. O script produz resultados detalhados por grupo e um resumo consolidado do diagnóstico antes da utilização dos ataques.

```
python codigo/diagnosticar_rnn_a06_residuos_normal_only.py
```

4.35 testar_rnn_a06_global_local.py

O programa `testar_rnn_a06_global_local.py` executa a avaliação global versus local do cenário A06 após o treinamento e o diagnóstico normal-only. O script reutiliza o mesmo checkpoint da RNN e os mesmos resíduos normais previamente produzidos para calibrar as estratégias global e local, garantindo que a comparação não envolva duas redes distintas. Os limiares são definidos com FPR nominal de 1% a partir da validação normal. Somente na etapa de teste são introduzidos os eventos sintéticos, sendo os rótulos alinhados ao último instante de cada janela temporal de 60 amostras. O programa grava tanto as métricas finais quanto os scores utilizados na comparação.

```
python codigo/testar_rnn_a06_global_local.py
```

4.36 treinar_rnn_fronteira_residuos_normais.py

O programa `treinar_rnn_fronteira_residuos_normais.py` treina uma RNN monolítica separadamente para cada cenário da bateria prospectiva da fronteira estrutural. O identificador do cenário é informado como argumento de linha de comando. O script verifica previamente o hash do protocolo congelado e os artefatos estruturais associados ao cenário, utiliza somente dados normais de treinamento e validação e mantém os mesmos hiperparâmetros utilizados no experimento A06: janela de 60 amostras, batch de 512, 64 unidades ocultas, máximo de 100 épocas, early stopping com paciência 5 e semente 42. Para cada cenário são gravados o melhor checkpoint, o histórico de treinamento, a ordem das features, os resíduos normais de treinamento e validação, os alinhamentos temporais e um resumo da execução.

```
python codigo/treinar_rnn_fronteira_residuos_normais.py A01
```

Para executar outro cenário, basta substituir `A01` pelo identificador correspondente, por exemplo `A02`, `A05`, `B03` ou `B06`. Não é necessário alterar o código-fonte entre os cenários.

4.37 diagnosticar_rnn_fronteira_residuos_normal_only.py

O programa `diagnosticar_rnn_fronteira_residuos_normal_only.py` aplica, para um cenário específico, o diagnóstico estrutural aos resíduos

normais produzidos pela RNN correspondente. O programa recebe o identificador do cenário como argumento, lê os resíduos de treinamento e validação, a ordem das features e a partição descoberta e estima os modelos global e locais de Mahalanobis com Ledoit–Wolf. Os limiares são determinados exclusivamente a partir da validação normal com FPR nominal de 1%. Como resultado são produzidos um arquivo detalhado por grupo e um resumo do cenário, ainda sem utilização dos ataques.

```
python
↪ codigo/diagnosticar_rnn_fronteira_residuos_normal_only.py
↪ A01
```

4.38 testar_rnn_fronteira_global_local.py

O programa `testar_rnn_fronteira_global_local.py` executa a avaliação final global versus local para um cenário da bateria prospectiva com RNN. Antes de abrir a etapa contendo ataques, o script verifica os registros de integridade dos artefatos de treinamento e dos diagnósticos normal-only, incluindo o checkpoint da RNN, os resíduos normais, a ordem das features e o diagnóstico estrutural previamente produzido. Caso algum desses arquivos não corresponda aos hashes registrados, a execução não deve prosseguir. Somente depois dessas verificações são carregados os eventos sintéticos e realizada a avaliação final. As estratégias global e local utilizam o mesmo checkpoint, os mesmos resíduos e a mesma partição previamente descoberta, diferindo apenas na forma como a evidência residual é agregada para produzir o score quadrático de Mahalanobis. Os limiares permanecem aqueles calibrados com a validação normal para FPR nominal de 1%. O programa produz os resultados finais e os scores utilizados em cada comparação.

```
python codigo/testar_rnn_fronteira_global_local.py A01
```

O mesmo comando deve ser repetido para os demais cenários prospectivos apenas substituindo o identificador informado como argumento.

Nota metodológica: o cenário A06 foi utilizado anteriormente como experimento inicial com RNN e, portanto, não é contabilizado como parte da bateria prospectiva. Na bateria posterior, os artefatos de treinamento e os diagnósticos produzidos apenas com dados normais foram concluídos e registrados antes da execução dos testes contendo ataques. Essa separação deve ser

preservada ao interpretar e reproduzir os resultados.

4.39 analisar_fronteira_grade.py

O programa `analisar_fronteira_grade.py` realiza uma análise consolidada dos resultados produzidos pela grade exploratória. A partir do resumo das condições avaliadas, o script compara o melhor desempenho global e local sob o diagnóstico com FPR limitado a 1%, incluindo separadamente os resultados obtidos com Mahalanobis, e organiza essas informações em tabelas destinadas a caracterizar regiões de comportamento da grade. Como saída são produzidas uma tabela principal, resumos por faixas e limites e matrizes que sintetizam as condições em que as estratégias locais apresentam vantagem sobre as alternativas globais. Essa rotina pertence à etapa de análise exploratória e não participa da calibração posterior dos experimentos confirmatórios.

```
python codigo/analisar_fronteira_grade.py
```

4.40 auditar_plausibilidade_univariada.py

O programa `auditar_plausibilidade_univariada.py` realiza uma auditoria adicional das perturbações empregadas no experimento piloto, examinando individualmente o comportamento das variáveis modificadas. O script compara o conjunto normal de treinamento, o teste normal de base, a versão do teste contendo ataques e a descrição dos eventos introduzidos. As saídas incluem resumos e informações detalhadas destinadas a verificar se as alterações sintéticas apresentam comportamento compatível com os valores observados nas respectivas variáveis, funcionando como controle de plausibilidade dos ataques antes de sua utilização nas análises experimentais.

```
python codigo/auditar_plausibilidade_univariada.py
```

4.41 gerar_normal_estrutural.py

O programa `gerar_normal_estrutural.py` contém a implementação do gerador normal utilizado como base para a construção dos conjuntos sintéticos estruturais. O modelo combina fatores latentes específicos de grupo, um fator global compartilhado e ruído individual, sendo parametrizado por α para

a contribuição intragrupo e β para a contribuição global. Na configuração presente no arquivo, α = 0,65 e β = 0,05, resultando teoricamente em correlação intragrupo aproximada de 0,70 e correlação entre grupos aproximada de 0,05 para as variáveis contínuas. O programa também cria parâmetros físicos determinísticos para os sensores, utiliza sementes fixas para tornar a geração reproduzível e pode produzir conjuntos normais de treinamento e validação, o mapa de referência das variáveis e um arquivo com os parâmetros utilizados na geração. A função responsável pela geração dos conjuntos normais é reutilizada diretamente por `gerar_cenarios_estruturais.py`, que importa `gerar_conjunto_normal` deste módulo e a emprega para construir os diferentes cenários da fronteira estrutural, variando de forma controlada os parâmetros α e β.

```
python codigo/gerar_normal_estrutural.py
```

4.42 testar_regressao_gerador_estrutural.py

O programa `testar_regressao_gerador_estrutural.py` constitui uma rotina de teste do gerador estrutural. Sua finalidade é verificar automaticamente propriedades esperadas da implementação e identificar alterações de comportamento que poderiam ser introduzidas por modificações posteriores no código do gerador ou em seus parâmetros. O script utiliza os parâmetros físicos previamente registrados e executa verificações sem produzir resultados de detecção de anomalias. Portanto, ele deve ser interpretado como teste de consistência do software experimental, e não como uma etapa adicional da avaliação dos métodos.

```
python codigo/testar_regressao_gerador_estrutural.py
```

Os programas descritos nesta seção abrangem a geração e auditoria dos dados sintéticos, a descoberta das partições, as etapas exploratória e confirmatória, a construção e análise da fronteira estrutural e os experimentos com RNN. Scripts auxiliares de auditoria e regressão são mantidos no repositório juntamente com os programas principais para permitir não apenas a repetição dos resultados, mas também a verificação das condições sob as quais os artefatos experimentais foram produzidos.

5 Integridade e reprodutibilidade

Neste documento, os termos *confirmatório* e *prospectivo* descrevem aspectos metodológicos distintos. A etapa confirmatória corresponde à repetição das condições previamente definidas em cinco realizações independentes dos dados, com o objetivo de verificar a estabilidade dos resultados observados anteriormente. Já a denominação *prospectiva*, utilizada na bateria com RNN, refere-se especificamente à ordem temporal do protocolo em relação à avaliação com ataques: o protocolo, o treinamento, os resíduos normais, o diagnóstico normal-only e seus registros de integridade foram concluídos antes da execução dos testes contendo perturbações. Assim, uma etapa ser confirmatória não implica que ela pertença à bateria prospectiva com RNN, e os dois termos não são utilizados como sinônimos.

5.1 Sementes e geração determinística

Os principais geradores utilizam sementes explícitas para permitir a reprodução das séries sintéticas, da ordem das variáveis e dos parâmetros físicos empregados nos experimentos. Sempre que treinamento, validação e teste representam conjuntos independentes, são utilizadas sementes distintas e previamente definidas. Os parâmetros físicos associados aos sensores também possuem semente própria e são reutilizados entre diferentes divisões dos dados, evitando que mudanças de escala sejam introduzidas artificialmente entre treinamento, validação e teste.

Na etapa com RNN, a semente utilizada pelo treinamento também é registrada no próprio script e nos resumos produzidos durante a execução. Os hiperparâmetros principais permanecem fixos entre os cenários da fronteira prospectiva, de forma que a mudança entre os experimentos corresponda à estrutura dos dados e não a uma nova busca de configuração da rede.

Observação: o uso das mesmas sementes permite reproduzir a sequência pseudoaleatória utilizada pelo código, mas diferenças de hardware, versões de bibliotecas ou implementação de determinadas operações numéricas podem produzir pequenas variações entre ambientes distintos. Por esse motivo, a reprodutibilidade deste projeto deve ser interpretada principalmente em termos de protocolo, dados, configuração e comportamento experimental, e não necessariamente como identidade binária de todos os valores numéricos produzidos em qualquer plataforma.

5.2 Congelamento dos protocolos

As etapas principais possuem arquivos de protocolo criados antes das avaliações correspondentes. Esses documentos registram parâmetros, condições, arquivos de entrada e decisões metodológicas relevantes, reduzindo a possibilidade de alterações silenciosas durante a execução posterior dos experimentos.

Entre os registros utilizados no projeto estão:

- `dados/protocolo_grade_exploratoria.json`;
- `dados/protocolo_confirmatorio.json`;
- `dados/fronteira_estrutural/protocolo_estrutural.json`;
- `dados/rnn_a06/protocolo_rnn_a06.json`;
- `dados/rnn_fronteira/protocolo_rnn_fronteira.json`.

Quando aplicável, o hash SHA-256 do protocolo é gravado separadamente. Os scripts responsáveis pelas etapas posteriores recalculam esses hashes antes da execução e interrompem o processamento quando o arquivo observado não corresponde ao valor esperado.

5.3 Separação entre dados normais e ataques

A distinção entre informação normal e informação de ataque é uma regra central do protocolo. A geração ou descoberta dos grupos utiliza dados normais de treinamento, enquanto os limiares empregados pelos detectores são calculados a partir de dados normais de validação. De forma semelhante, o diagnóstico estrutural normal-only é calculado antes da introdução das perturbações utilizadas para avaliar empiricamente o comportamento associado às condições estruturais identificadas pelo diagnóstico.

Os ataques sintéticos são utilizados posteriormente para medir o desempenho dos detectores e avaliar empiricamente as hipóteses formuladas nas etapas anteriores. Eles não são utilizados para formar os grupos, estimar as covariâncias normais, calibrar os limiares principais ou calcular o diagnóstico estrutural empregado prospectivamente.

As análises explicitamente retrospectivas presentes no repositório, como o casamento de FPR no conjunto de teste e a análise direcional dos cenários

B05 e B06, são identificadas separadamente neste documento e não devem ser confundidas com o protocolo de avaliação principal.

5.4 Integridade da etapa confirmatória

Na confirmação independente são congelados previamente as condições experimentais, os eventos de ataque e a ordem das variáveis utilizadas na construção das perturbações. Os arquivos correspondentes possuem registros SHA-256 e são verificados novamente pelo programa de execução confirmatória antes do início das avaliações.

Entre os registros associados a essa etapa estão:

- `dados/sha256_protocolo_confirmatorio.txt`;
- `dados/sha256_eventos_confirmatorios.txt`;
- `dados/condicoes_confirmatorias.csv`;
- `dados/eventos_confirmatorios_base.csv`;
- `dados/ordem_features_confirmatorias.csv`.

As cinco realizações independentes são produzidas somente após a definição dessas condições. Cada realização possui seu próprio conjunto normal de treinamento, validação e teste de base, permitindo avaliar se o comportamento encontrado inicialmente permanece estável em novas realizações dos dados sintéticos.

5.5 Integridade da fronteira estrutural

A grade da fronteira estrutural e seu protocolo são registrados antes da geração dos cenários. O arquivo `dados/fronteira_estrutural/sha256_protocolo_estrutural.txt` preserva os hashes associados a essa definição experimental. A partir dela são construídos os diferentes cenários e seu manifesto, mantendo separadas a definição da estrutura experimental e as análises realizadas posteriormente.

A partição descoberta em cada cenário é produzida a partir dos respectivos dados normais antes das avaliações com ataques. As análises que utilizam diretamente a partição de referência do gerador são armazenadas separadamente e identificadas neste documento como análises de referência.

5.6 Congelamento da bateria prospectiva com RNN

A bateria prospectiva com RNN utiliza um nível adicional de controle de integridade. Depois do registro do protocolo, do treinamento das redes e da conclusão dos diagnósticos normal-only, os artefatos de treinamento e os diagnósticos correspondentes são registrados em manifestos SHA-256 antes da execução dos testes contendo ataques.

Os principais manifestos são:

- `dados/rnn_fronteira/sha256_protocolo_rnn_fronteira.txt`;
- `dados/rnn_fronteira/sha256_artefatos_treino.txt`;
- `dados/rnn_fronteira/sha256_diagnosticos_normal_only.txt`;
- `dados/rnn_fronteira/sha256_resultados_testes.txt`.

O manifesto de artefatos de treinamento registra arquivos como os checkpoints das RNNs, os resíduos normais de treinamento e validação e a ordem das features utilizada por cada cenário. O manifesto dos diagnósticos registra as saídas produzidas exclusivamente a partir desses resíduos normais. O script de teste verifica esses registros antes de utilizar os ataques, impedindo que um checkpoint, vetor de resíduos ou diagnóstico alterado seja utilizado silenciosamente na avaliação final.

O arquivo `dados/rnn_fronteira/sha256_resultados_testes.txt` é produzido somente após a execução dos testes e registra os resultados e scores finais. Sua função é preservar o estado dos resultados obtidos, e não participar da definição ou calibração dos detectores.

Na execução registrada, o manifesto dos artefatos de treinamento contém 44 entradas, correspondentes a quatro arquivos para cada um dos onze cenários prospectivos: o melhor checkpoint da RNN, os resíduos normais de treinamento, os resíduos normais de validação e a ordem das features. O manifesto dos diagnósticos normal-only contém 11 entradas, uma para cada cenário, enquanto o manifesto final dos testes contém 22 entradas, correspondentes a um arquivo de resultados e um arquivo de scores para cada cenário.

As quantidades podem ser verificadas diretamente por:

```
wc -l \
  dados/rnn_fronteira/sha256_artefatos_treino.txt \
```

```
dados/rnn_fronteira/sha256_diagnosticos_normal_only.txt \
dados/rnn_fronteira/sha256_resultados_testes.txt
```

Para a execução registrada neste repositório, são esperadas respectivamente 44, 11 e 22 entradas.

5.7 Status do cenário A06

O cenário A06 possui protocolo e artefatos próprios em `dados/rnn_a06/` e `resultado/rnn_a06/`. Esse cenário foi executado antes da definição da bateria prospectiva e serviu como experimento inicial para verificar a aplicação do diagnóstico no espaço dos resíduos produzidos pela RNN.

Por essa razão, A06 é mantido no repositório para rastreabilidade e reprodução, mas não deve ser contabilizado como integrante da validação prospectiva posterior. A bateria prospectiva é composta pelos onze cenários A01–A05 e B01–B06, cujos artefatos normal-only foram concluídos e registrados antes da abertura dos respectivos testes com ataques.

5.8 Sequência de integridade

A sequência utilizada na bateria prospectiva com RNN pode ser representada de forma resumida por:

$$\text{protocolo} \rightarrow \text{treinamento normal} \rightarrow \text{resíduos normais} \rightarrow \text{diagnóstico normal-only} \rightarrow \text{congelam} \tag{20}$$

Essa sequência deve ser preservada por qualquer tentativa de reprodução que pretenda manter os mesmos controles metodológicos do experimento original.